Final Capstone Project

From Raw Feed to Recommendation

1. What this project is

Over nine days you have built the pieces separately. You have retrieved and joined telecom data in SQL, cleaned a messy file in pandas, built an ETL that repairs bad data instead of deleting it, scheduled it so it runs without you, trained a churn classifier with the leaking columns removed, read an ARPU regression through its residuals, forecast traffic per cell, loaded a live KPI feed that survives being loaded twice, taken an alert rule from a hundred pages a week down to a handful, found subscriber segments and fraud cases nobody had labelled, and built two language models that read what customers write.

This project asks one thing of you that none of those exercises did: make them one system. The mini-capstone taught you two new techniques from scratch and asked you to apply them. Nothing in this brief is new. Every method you need has already been run, either in class or as a home task. What is new is the join - the point where a model output becomes a dashboard row, where a dashboard row becomes a rupee figure, and where a rupee figure becomes something a manager will actually approve.

> **IMPORTANT** Read this whole brief once, as a team, before you choose a track. The code here is a scaffold to adapt, not to paste blindly. The marks are in the decisions you make and the way you explain them, not in the code running.

The data is new

Your Capstone Data folder holds a fresh extract - a different set of subscribers, a different week of network readings, a different message archive. The column names will look familiar because the schema is the same one you have worked with all fortnight. The numbers are not. Every answer you reach has to be computed from these files; nothing carries over from Exercise 5, Exercise 8 or the mini-capstone.

> **WHAT THAT MEANS FOR YOU** Every count printed in this brief was produced by running this exact code against the shipped files. If your number differs from the one printed here, something in your steps differs - find it before you move on. That is not a formality; two of the numbers in this brief change depending on the order you clean in, and the brief tells you when.

2. Working as a team of five

One notebook and one deck per team. How you divide the work is yours to decide, but the project splits naturally into five areas of ownership.

Role	Owns	Feeds into
Data & pipeline lead	The integration script, the data dictionary, keeping the notebook running end to end	Everyone depends on this
Modelling lead	The model, the honest baseline, the validation	The central finding

Role	Owns	Feeds into
Operations lead	The dashboard, and the alerting or routing policy where the track calls for one	The live demonstration
Business analyst	The costed comparison, the assumptions, the SIAV close	The recommendation slides
Presentation lead	The deck, the one-page summary, rehearsing the talk	The whole presentation

Every member must be able to explain the whole project, not only their own part. In the question period we choose who answers. The two easiest ways to lose marks as a team are a notebook only one person understands, and a deck where the technical and business halves clearly never spoke to each other.

3. Your files

File	What it holds	Used by
capstone_subscribers.csv	One row per subscriber - profile, six-month behaviour summary, churn outcome	Tracks A, C
capstone_usage_monthly.csv	One row per subscriber per month, six months of usage and revenue	Track A
capstone_network_kpi.csv	15-minute cell readings across five days, eighteen cells	Track B
capstone_maintenance_windows.csv	Planned outages with change tickets	Track B
capstone_cell_traffic_monthly.csv	Thirty months of traffic and engineered capacity per cell	Track B
capstone_messages.csv	Customer messages with the true sentiment attached	Track C
capstone_tickets.csv	Support tickets with the team that handled them	Track C

A separate Capstone Data Dictionary explains every column, and - more importantly - when each value is recorded. Read it before you model. Two of the columns in the subscriber file cannot honestly be used to predict churn, and the dictionary is where you will find out why.

4. Part 1 - Build the governed dataset

Every team does this part, whichever track you choose. It is the foundation the rest of your project stands on, and it is worth doing carefully rather than quickly.

4.1 Count before you clean

Never load a file you have not counted. This takes thirty seconds and saves an hour of confusion later, because once you have started cleaning you can no longer tell whether a number is a property of the data or a side effect of what you did to it.

CODE - RUN THIS FIRST

```
import pandas as pd, numpy as np

DATA = "."                    # folder holding your capstone CSV files
s = pd.read_csv(f"{DATA}/capstone_subscribers.csv")

print("rows in file         :", len(s))
print("distinct customer_id  :", s.customer_id.nunique())
print("duplicate customer_id :", s.duplicated(subset=["customer_id"]).sum())
print("distinct region values:", s.region.nunique())
print("missing avg_monthly_gb:", s.avg_monthly_gb.isna().sum())
print("missing days_since_last_recharge:", s.days_since_last_recharge.isna().sum())
```

EXPECTED OUTPUT

```
rows in file         : 3222
distinct customer_id  : 3200
duplicate customer_id : 22
distinct region values: 18
missing avg_monthly_gb: 99
missing days_since_last_recharge: 58
```

LOOK AT THE REGION COUNT There are 18 distinct region values in a file that covers six cities. That is not eighteen cities - it is six cities spelled several different ways, with stray capitals and leading spaces. If you group by region before fixing this, every regional total you produce will be wrong and nothing will warn you.

4.2 Repair, do not delete

The rule from Exercise 3 applies here and will be checked. A row with a bad value is repaired and kept, with the repair recorded. Dropping rows quietly changes the population you are drawing conclusions about.

CODE

```
# 1) text standardisation - do this BEFORE any groupby
s["region"] = s.region.str.strip().str.title()
print("region values after cleaning:", s.region.nunique(), "->", sorted(s.region.unique()))

# 2) de-duplicate on the business key, keeping the first occurrence
before = len(s)
s = s.drop_duplicates(subset=["customer_id"], keep="first").copy()
print(f"removed {before - len(s)} duplicate subscriber rows, {len(s)} remain")

# 3) fill numeric gaps with a median and RECORD that you did it
for c in ["avg_monthly_gb", "days_since_last_recharge"]:
    s[c + "_was_missing"] = s[c].isna().astype(int)
    s[c] = s[c].fillna(s[c].median())
print("rows flagged as imputed:", s[["avg_monthly_gb_was_missing",
      "days_since_last_recharge_was_missing"]].sum().to_dict())
```

EXPECTED OUTPUT

```
region values after cleaning: 6 -> ['Bengaluru', 'Chennai', 'Delhi', 'Hyderabad', 'Kolkata', 'Mumbai']
removed 22 duplicate subscriber rows, 3200 remain
rows flagged as imputed: {'avg_monthly_gb_was_missing': 99, 'days_since_last_recharge_was_missing': 58}
```

WHY THE _was_missing FLAG MATTERS A median-filled value looks exactly like a measured one once it is in the frame. The flag column lets your model treat "we did not observe this" as its own signal, and lets you answer the question we will certainly ask: how much of your finding rests on numbers you invented?

4.3 Prove your pipeline can be run twice

This is the Exercise 8 discipline, and it is the single check that separates a system from a one-off analysis. Wrap your cleaning in a function, run it twice, and assert that nothing moved.

CODE - LEAVE THIS IN YOUR NOTEBOOK

```
OUT_PATH = "capstone_integrated.csv"

def run_pipeline():
    df = pd.read_csv(f"{DATA}/capstone_subscribers.csv")
    df["region"] = df.region.str.strip().str.title()
    df = df.drop_duplicates(subset=["customer_id"], keep="first")
    # ... the rest of your transformations, and your track-specific joins ...
    df.to_csv(OUT_PATH, index=False)
    return len(df)

first  = run_pipeline()
second = run_pipeline()
print(f"first run: {first} rows   second run: {second} rows")
assert first == second, f"pipeline is not idempotent: {first} -> {second}"
print("pipeline is idempotent")
```

EXPECTED OUTPUT

```
first run: 3200 rows   second run: 3200 rows
pipeline is idempotent
```

> **LEARNING POINT** A pipeline that cannot be run twice cannot be scheduled, and a pipeline that cannot be scheduled is an analysis rather than a system. Those three lines are the difference, and they cost you nothing.

4.4 State the grain

Write one sentence in your notebook: one row per what? If your team cannot answer that in five words, your table is not finished. Every downstream number - every average, every count, every rupee figure - depends on getting this right, and it is the first question we will ask you on Day 10.

5. Part 2

Track A - Churn prediction and retention strategy

The business question: which subscribers are we about to lose, which of them is worth spending money to keep, and what should we spend it on?
Files: capstone_subscribers.csv and capstone_usage_monthly.csv

A1 Fold six months of behaviour into one row per subscriber

The subscriber file already carries averages. The monthly file carries the shape of those averages over time - whether a subscriber is using more than they were, or less. A subscriber whose data use halved over six months looks identical to a steady one in the averages, and completely different in the trend.

CODE

```
u = pd.read_csv(f"{DATA}/capstone_usage_monthly.csv")
print("rows in file:", len(u), " duplicate (customer_id, month):",
      u.duplicated(subset=["customer_id","month"]).sum())

u = u.drop_duplicates(subset=["customer_id","month"], keep="first").copy()
print("after de-duplication:", len(u), "rows across", u.month.nunique(), "months")

agg = u.groupby("customer_id").agg(
    months_seen      = ("month", "nunique"),
    tot_gb           = ("data_gb", "sum"),
    tot_voice        = ("voice_min", "sum"),
    tot_intl         = ("intl_min", "sum"),
    tot_rev          = ("revenue_inr", "sum"),
    zero_rev_months  = ("revenue_inr", lambda x: int((x == 0).sum())),
    fails            = ("failed_payment_count", "sum"),
).reset_index()

# the trend feature: recent three months against the first three
late  = u[u.month >= "2026-05"].groupby("customer_id").data_gb.mean().rename("gb_late")
early = u[u.month <  "2026-05"].groupby("customer_id").data_gb.mean().rename("gb_early")
agg = agg.merge(late, on="customer_id").merge(early, on="customer_id")
agg["gb_trend"] = agg.gb_late / agg.gb_early.clip(lower=0.01)

df = s.merge(agg, on="customer_id", how="left")
df["complaint_intensity"] = df.complaints_6m / (df.tenure_months / 6).clip(lower=1)
print("merged frame:", df.shape)
```

EXPECTED OUTPUT

```
rows in file: 19238  duplicate (customer_id, month): 38
after de-duplication: 19200 rows across 6 months
merged frame: (3200, 32)
```

COMPLAINT INTENSITY, AGAIN Two complaints from a two-month subscriber and two from a five-year subscriber are not the same event. You built this ratio in Exercise 5 for exactly that reason; it belongs here too.

A2 The leakage audit - do it again, do not assume it

Two columns in the subscriber file fail the day-before test: would this value exist, with this value, the day before the subscriber churned? Work through every column and record a verdict. Then prove the leak rather than asserting it.

CODE

```
# evidence 1 - the offer column is a consequence, not a cause
print(df.groupby("churn").retention_offer_sent.mean().round(3))

# evidence 2 - total_charges is arpu multiplied by tenure
implied = df.total_charges / df.tenure_months.clip(lower=1)
print("correlation of implied ARPU with arpu:", round(implied.corr(df.arpu), 4))
```

EXPECTED OUTPUT

```
churn
0    0.051
1    0.843

correlation of implied ARPU with arpu: 0.9995
```

Read those two numbers as a business fact rather than a statistic. A retention offer went to 84 per cent of the subscribers who left and 5 per cent of those who stayed - because the offer was triggered by a churn signal somebody had already spotted. Feeding it back into a churn model is asking the model to predict the past.

CAUTION total_charges is not bad data. It is perfectly valid for revenue reporting. It is wrong for this target, because it contains the answer. Leakage is a property of the pairing of feature and target, not of the column on its own - which is why the audit has to be re-run whenever the target changes.

A3 Measure what the leak is worth

Train once with the leaking columns in, once with them out, and record both numbers. The gap is the evidence for your write-up and the single most persuasive slide you will have.

CODE

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score

def build_and_score(drop_cols, model=None):
    X = df.drop(columns=["churn", "customer_id"] + drop_cols)
    y = df["churn"]
    num = X.select_dtypes(include=np.number).columns.tolist()
    cat = X.select_dtypes(exclude=np.number).columns.tolist()
    pre = ColumnTransformer([
        ("n", Pipeline([("i", SimpleImputer(strategy="median")),
                        ("s", StandardScaler())]), num),
        ("c", Pipeline([("i", SimpleImputer(strategy="most_frequent")),
                        ("o", OneHotEncoder(handle_unknown="ignore"))]), cat)])
    Xtr, Xte, ytr, yte = train_test_split(
        X, y, test_size=0.25, stratify=y, random_state=42)
    clf = model if model is not None else RandomForestClassifier(
        n_estimators=300, min_samples_leaf=3, random_state=42)
    pipe = Pipeline([("pre", pre), ("clf", clf)]).fit(Xtr, ytr)
    proba = pipe.predict_proba(Xte)[:, 1]
    return roc_auc_score(yte, proba), proba, yte

LEAKY = ["retention_offer_sent", "total_charges"]
auc_leak,  _, _        = build_and_score([])
auc_clean, proba, yte  = build_and_score(LEAKY)
print(f"ROC-AUC with the leak : {auc_leak:.3f}")
print(f"ROC-AUC without it    : {auc_clean:.3f}")
print(f"the leak was worth    : {auc_leak - auc_clean:.3f}")
```

EXPECTED OUTPUT

```
ROC-AUC with the leak : 0.946
ROC-AUC without it    : 0.729
the leak was worth    : 0.217
```

THIS IS THE SLIDE A model scoring 0.946 would have been presented as a success, deployed, and would have predicted nothing at all in production - because on the day you need a score, nobody has sent the retention offer yet. 0.729 is the honest number. Put both on one slide and explain the gap in one sentence.

A4 Compare three classifiers on identical splits

CODE

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import precision_score, recall_score, accuracy_score

models = {
  "Logistic regression": LogisticRegression(max_iter=2000),
  "Decision tree":       DecisionTreeClassifier(max_depth=6, random_state=42),
  "Random forest":       RandomForestClassifier(n_estimators=300,
                         min_samples_leaf=3, random_state=42),
}
rows = []
for name, m in models.items():
    a, pr, yt = build_and_score(LEAKY, m)
    p05 = (pr >= 0.5).astype(int)
    rows.append({"model": name, "roc_auc": round(a, 3),
                 "precision": round(precision_score(yt, p05, zero_division=0), 3),
                 "recall":    round(recall_score(yt, p05), 3),
                 "accuracy":  round(accuracy_score(yt, p05), 3)})
pd.DataFrame(rows)
```

Model	ROC-AUC	Precision	Recall	Accuracy
Logistic regression	0.742	0.676	0.240	0.790
Decision tree	0.643	0.507	0.182	0.761
Random forest	0.729	0.739	0.089	0.774

WHAT YOU WILL SEE All three land in the same neighbourhood, and every one of them has recall well under a third at the default threshold. That is the honest result on this data. Do not tune anything yet - step A5 tells you what to tune for, and tuning before you know that is guessing.

A5 Compare against the do-nothing baseline

CODE

```
baseline = 1 - yte.mean()
print("predict nobody churns - accuracy :", round(baseline, 3))
print("random forest at 0.5  - accuracy :",
      round(((proba >= 0.5).astype(int) == yte).mean(), 3))
```

EXPECTED OUTPUT

```
predict nobody churns - accuracy : 0.760
random forest at 0.5  - accuracy : 0.774
```

LEARNING POINT Predicting that nobody churns is right 76.0 per cent of the time and is worth nothing to anybody. Your model beats it by a little over one percentage point on accuracy. If accuracy were your headline metric you would have to report this project as a failure - which is precisely why accuracy is the wrong metric for an imbalanced problem, and why the next step exists.

A6 Choose a threshold from the cost of being wrong

The figures from Exercise 5 carry forward: a subscriber lost is worth about ₹5,880 in lifetime value, a retention contact costs about ₹300. That is a ratio of roughly 19.6 to one, and it is the only thing that should decide your threshold.

CODE

```
from sklearn.metrics import confusion_matrix
COST_FN, COST_FP = 5880, 300

rows = []
for t in np.arange(0.05, 0.95, 0.05):
    pred = (proba >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(yte, pred).ravel()
    rows.append({"threshold": round(t, 2),
                 "precision": round(precision_score(yte, pred, zero_division=0), 3),
                 "recall":    round(recall_score(yte, pred), 3),
                 "flagged":   int(pred.sum()),
                 "expected_cost": fn * COST_FN + fp * COST_FP})
sweep = pd.DataFrame(rows)
sweep.sort_values("expected_cost").head(4)
```

Threshold	Precision	Recall	Flagged	Expected cost
0.05	0.241	1.000	796	₹1,81,200
0.10	0.256	0.990	742	₹1,77,360
0.15	0.282	0.932	635	₹2,13,240
0.20	0.322	0.833	497	₹2,89,260
0.30	0.444	0.552	239	₹5,45,580
0.50	0.739	0.089	23	₹10,30,800

The cheapest threshold on this test set is 0.10, at an expected cost of ₹1,77,360. The software default of 0.50 costs ₹10,30,800 - nearly six times as much, on identical predictions. Nothing about the model changed; only the line you drew through it.

CODE

```
# the closed-form optimum, for comparison with your sweep
print("theoretical optimum:", round(COST_FP / (COST_FP + COST_FN), 4))
```

EXPECTED OUTPUT

```
theoretical optimum: 0.0485
```

THE UNCOMFORTABLE PART At 19.6 to one the arithmetic wants you to contact almost everybody - 742 of 800 subscribers in the test set. That is a real answer, and it is telling you something true: at these costs the binding constraint is not the model, it is how many calls your retention team can actually make. Say that out loud. Then decide your threshold against the team's capacity as well as the cost ratio, and show both numbers.

A7 Segment, and attach risk to each segment

You did this in the mini-capstone. The method is the same; the data is not. Cluster on behaviour, not on decisions the business already made - leave region and plan_type out of the clustering and use them afterwards to describe what you found.

CODE

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

feats = ["avg_monthly_gb", "avg_voice_min", "tenure_months",
         "arpu", "complaints_6m", "days_since_last_recharge"]
X = SimpleImputer(strategy="median").fit_transform(df[feats])
X = StandardScaler().fit_transform(X)          # never skip this

for kk in range(2, 9):
    km = KMeans(n_clusters=kk, n_init=10, random_state=42).fit(X)
    sil = silhouette_score(X, km.labels_, sample_size=2000, random_state=1)
    print(f"k={kk}  inertia={km.inertia_:9.1f}  silhouette={sil:.3f}")
```

EXPECTED OUTPUT

```
k=2  inertia=  15575.2  silhouette=0.238
k=3  inertia=  13853.9  silhouette=0.235
k=4  inertia=  12290.8  silhouette=0.204
k=5  inertia=  10913.5  silhouette=0.212
k=6  inertia=   9710.8  silhouette=0.205
k=7  inertia=   9044.2  silhouette=0.176
k=8  inertia=   8674.9  silhouette=0.158
```

WHAT YOU WILL SEE Inertia falls smoothly with no sharp elbow, and silhouette drifts downward rather than peaking. This is normal for real subscriber data. Do not invent a clean elbow that is not there - report honestly what the numbers show, then choose k on the commercial test instead: could the retention team genuinely run this many different campaigns? Four is realistic. Twelve is not.

CODE

```
K = 4                                  # your choice - change it and justify it
km = KMeans(n_clusters=K, n_init=10, random_state=42).fit(X)
df["segment"] = km.labels_

profile = df.groupby("segment").agg(
    size           = ("customer_id", "count"),
    mean_arpu      = ("arpu", "mean"),
    mean_tenure    = ("tenure_months", "mean"),
    mean_gb        = ("avg_monthly_gb", "mean"),
    mean_complaints = ("complaints_6m", "mean"),
    churn_rate     = ("churn", "mean")).round(2)
profile["share_pct"] = (profile["size"] / len(df) * 100).round(1)
profile
```

Segment	Size	Share %	Mean ARPU	Mean tenure	Mean GB	Complaints	Churn rate
0	392	12.2	239.49	26.5	7.37	3.37	0.40

Segment	Size	Share %	Mean ARPU	Mean tenure	Mean GB	Complaints	Churn rate
1	1628	50.9	204.37	21.7	5.70	0.46	0.23
2	661	20.7	406.68	24.0	18.76	0.69	0.21
3	519	16.2	249.96	25.5	6.57	0.57	0.19

One segment stands out sharply: segment 0 churns at 40 per cent against a book average of 24.1 per cent, and it is defined by complaints rather than by spend. It is 12.2 per cent of the base. That is your target group, and the number you build your recommendation on.

> **LEARNING POINT** If two segments would receive the same campaign, you have too many. Merge them and say so in your write-up - recognising that is a finding, not a failure.

A8 Find what nobody labelled

Hidden in the monthly file are subscribers whose behaviour is wrong - fraud and billing faults. Nobody has flagged them. Three patterns are present; you are not told which subscribers they belong to or how many there are.

Pattern	What it looks like in the data
Bypass / SIM-box fraud	Enormous international minutes, almost no data, and a small bill for all that traffic
Subscription fraud	Usage climbs month on month, then revenue drops to zero and payments start failing
Revenue leakage	A month with genuine usage billed at exactly zero - a billing fault, not fraud

CODE

```
from sklearn.ensemble import IsolationForest

u["intl_share"]     = u.intl_min / (u.voice_min + u.intl_min + 1)
u["revenue_per_gb"] = u.revenue_inr / u.data_gb.clip(lower=0.01)
u["zero_revenue"]   = (u.revenue_inr == 0).astype(int)
u["data_to_voice"]  = u.data_gb / u.voice_min.clip(lower=1)

feat = ["data_gb","voice_min","intl_min","sms_count","revenue_inr",
        "failed_payment_count","intl_share","revenue_per_gb","data_to_voice"]
Xa = StandardScaler().fit_transform(
        u[feat].replace([np.inf, -np.inf], 0).fillna(0))

for c in [0.005, 0.01, 0.02]:
    iso = IsolationForest(contamination=c, n_estimators=300,
                          random_state=42).fit(Xa)
    u[f"flag_{c}"]  = (iso.predict(Xa) == -1).astype(int)
    u[f"score_{c}"] = -iso.score_samples(Xa)
    n = int(u[f"flag_{c}"].sum())
    print(f"contamination {c:<6} flagged {n:>4} rows"
          f"  ~{n/26:.1f}/week   cost Rs {n*450:,}")
```

EXPECTED OUTPUT

```
contamination 0.005  flagged   96  ~3.7/week   cost Rs 43,200
contamination 0.01   flagged  192  ~7.4/week   cost Rs 86,400
contamination 0.02   flagged  384  ~14.8/week   cost Rs 1,72,800
```

Now make it a decision rather than a setting. Assume a two-person revenue-assurance team that can genuinely investigate about eight cases a week, at roughly ₹450 of analyst time per case, over the six-month window in the data. Which contamination level fits that capacity, what do you give up by choosing it, and what would you need in order to justify the more sensitive setting?

CAUTION Contamination is not discovered in the data. It is a decision about how many cases you are willing to investigate, dressed up as a parameter. And a flag you have not looked at is not evidence - pull the top twenty, attach subscriber context, and classify each one by hand into one of the three patterns or as an ordinary heavy user who is not a problem at all. Never call a flag a conclusion; it is a case for investigation.

A9 Rank on risk and value together

ONE THING TO WATCH The scores you swept in A6 came from a model that never saw the test fold. To build a call list you need a score for every subscriber, so refit on the whole frame - but never report a metric from that refit model. Scoring and evaluating are two different jobs, and mixing them is how an honest team accidentally reports a fabricated number.

A churn list sorted by risk alone tells the retention team to spend the same effort on a ₹120-a-month subscriber as on an ₹800 one. Combine the two before you hand anybody a call list.

CODE

```
# score every subscriber, not just the held-out fold
X_all = df.drop(columns=["churn", "customer_id"] + LEAKY)
full  = Pipeline([("pre", pre), ("clf", RandomForestClassifier(
          n_estimators=300, min_samples_leaf=3, random_state=42))])
full.fit(X_all, df["churn"])              # refit on everything, for scoring only
df["risk"] = full.predict_proba(X_all)[:, 1]

df["expected_loss"] = df.risk * df.arpu * 12      # a year of revenue at risk
call_list = df.sort_values("expected_loss", ascending=False)
print(call_list[["customer_id","segment","risk","arpu","expected_loss"]].head(10))

# how different is this from ranking on risk alone?
top_risk = set(df.nlargest(300, "risk").customer_id)
top_value = set(df.nlargest(300, "expected_loss").customer_id)
print("overlap between the two top-300 lists:", len(top_risk & top_value))
```

THE QUESTION WE WILL ASK Your model scores a subscriber at 0.62. Walk us through what happens next, physically, inside the business. Who sees that number, when do they see it, and what do they do about it?

Track B - Network quality and revenue optimisation

The business question: which cells are degrading, which of that degradation is actually costing revenue, and where should next quarter's capacity spend go?
Files: capstone_network_kpi.csv, capstone_maintenance_windows.csv, capstone_cell_traffic_monthly.csv, and capstone_subscribers.csv for regional ARPU.

B1 Profile the feed before you load it

CODE

```
kp = pd.read_csv(f"{DATA}/capstone_network_kpi.csv")
print("rows in file            :", len(kp))
print("distinct cells          :", kp.cell_id.nunique())
print("duplicate (cell_id, ts)  :",
      kp.duplicated(subset=["cell_id","reading_ts"]).sum())
print("blank latency_ms        :", kp.latency_ms.isna().sum())
print("negative throughput_mbps :", (kp.throughput_mbps < 0).sum())
print("rows with downtime > 0   :", (kp.downtime_min > 0).sum())
```

EXPECTED OUTPUT

```
rows in file            : 8681
distinct cells          : 18
duplicate (cell_id, ts)  : 41
blank latency_ms        : 97
negative throughput_mbps : 9
rows with downtime > 0   : 27
```

B2 Classify the blanks before you decide what to do with them

STOP AND READ THIS BEFORE WRITING ANY CODE You have blank latency values. The instinct is to treat them all the same way - impute them, drop them, or raise a ticket against the collector. That instinct is wrong, and this step exists to prove it. Some of those blanks are the correct value.

CODE

```
kc = kp.drop_duplicates(subset=["cell_id","reading_ts"], keep="first").copy()
kc["ts"] = pd.to_datetime(kc.reading_ts)
print("after de-duplication:", len(kc), "rows")

mw = pd.read_csv(f"{DATA}/capstone_maintenance_windows.csv")
mw["window_start"] = pd.to_datetime(mw.window_start)
mw["window_end"]   = pd.to_datetime(mw.window_end)

in_maint = pd.Series(False, index=kc.index)
for _, w in mw.iterrows():
    in_maint |= ((kc.cell_id == w.cell_id) &
                 (kc.ts >= w.window_start) & (kc.ts < w.window_end))

blank   = kc.latency_ms.isna()
outage  = blank & ~in_maint & (kc.downtime_min > 0)
print("blank total                        :", int(blank.sum()))
print("blank inside a maintenance window  :", int((blank & in_maint).sum()))
print("blank during an unplanned outage   :", int(outage.sum()))
print("blank with no explanation at all   :",
      int((blank & ~in_maint & ~outage).sum()))
```

EXPECTED OUTPUT

```
after de-duplication: 8640 rows
blank total                        : 90
blank inside a maintenance window  : 20
blank during an unplanned outage   : 7
blank with no explanation at all   : 63
```

WHAT THIS MEANS The 20 blanks inside a maintenance window are not missing data. An engineer switched the cell off holding a change ticket. There was no traffic, so there was no latency to measure, and a NULL is the honest record of that. Imputing them invents a measurement of something that was not happening.
The 7 blanks during the unplanned outage are also not collector faults - they are the outage itself, and they are the most important rows in the file.
Only the remaining 63 are genuine collector losses, and only those are worth a ticket.
Write all four numbers down; you will be asked for them.

B3 Repair only what is genuinely broken

CODE

```
# negative throughput is a counter rollover, not a measurement of anything
neg = kc.throughput_mbps < 0
kc.loc[neg, "throughput_mbps"] = None
print("negative throughput nulled:", int(neg.sum()))
print("blanks left untouched     :", int(kc.latency_ms.isna().sum()))
```

EXPECTED OUTPUT

```
negative throughput nulled: 9
blanks left untouched     : 90
```

ORDER MATTERS, AND IT WILL BITE YOU The blank counts in B2 were taken after de-duplication. Take them before, and you get different numbers, because some blanks were sitting on duplicated rows. Neither answer is wrong - but only one of them is the answer to the question you were asked. State the grain, state the operation, then count again.

B4 Load it so a second load changes nothing

CODE

```
import sqlite3
con = sqlite3.connect("capstone_rt.db")
con.executescript("""
CREATE TABLE IF NOT EXISTS kpi_readings (
  reading_ts        TEXT NOT NULL,
  cell_id           TEXT NOT NULL,
  region            TEXT NOT NULL,
  latency_ms        REAL,
  throughput_mbps   REAL,
  downtime_min      INTEGER NOT NULL DEFAULT 0,
  drop_call_rate_pct REAL,
  active_users      INTEGER,
  UNIQUE (cell_id, reading_ts)
);
CREATE INDEX IF NOT EXISTS ix_cell_ts ON kpi_readings (cell_id, reading_ts);
""")
con.commit()

cols = ["reading_ts","cell_id","region","latency_ms","throughput_mbps",
        "downtime_min","drop_call_rate_pct","active_users"]
recs = kc[cols].astype(object).where(pd.notna(kc[cols]), None).values.tolist()

def load(rows):
    con.executemany("INSERT OR IGNORE INTO kpi_readings "
        "(reading_ts, cell_id, region, latency_ms, throughput_mbps, "
        "downtime_min, drop_call_rate_pct, active_users) "
        "VALUES (?,?,?,?,?,?,?,?)", rows)
    con.commit()

load(recs)
print("after first load :", con.execute(
      "SELECT COUNT(*) FROM kpi_readings").fetchone()[0])
load(recs)
print("after second load:", con.execute(
      "SELECT COUNT(*) FROM kpi_readings").fetchone()[0])
```

EXPECTED OUTPUT

```
after first load : 8640
after second load: 8640
```

THE UNIQUE CONSTRAINT IS THE WHOLE POINT Without it, every re-run of your loader doubles the row count and every count-based alert you build afterwards is wrong by a factor nobody notices. A live feed re-sends readings. A pipeline that cannot absorb that safely produces silently wrong numbers.

B5 The same incident at two grains

There is a congestion incident on CELL-DEL-03 on the evening of 3 August. Look at it two ways.

CODE - 15-MINUTE GRAIN, THE TRUE PEAK

```
q = """SELECT reading_ts, latency_ms FROM kpi_readings
      WHERE cell_id = 'CELL-DEL-03' AND reading_ts LIKE '2026-08-03%'
      ORDER BY latency_ms DESC LIMIT 3"""
for r in con.execute(q): print(r)
```

EXPECTED OUTPUT

```
('2026-08-03 20:00:00', 117.25)
('2026-08-03 20:45:00', 117.08)
('2026-08-03 21:30:00', 116.37)
```

CODE - 60-MINUTE GRAIN

```
q = """SELECT substr(reading_ts,1,13) || ':00' AS window_start,
            ROUND(AVG(latency_ms),2) AS avg_latency,
            ROUND(MAX(latency_ms),2) AS peak_latency,
            COUNT(*) AS readings
      FROM kpi_readings
      WHERE cell_id = 'CELL-DEL-03' AND reading_ts LIKE '2026-08-03%'
      GROUP BY window_start ORDER BY avg_latency DESC LIMIT 3"""
for r in con.execute(q): print(r)
```

EXPECTED OUTPUT

```
('2026-08-03 21:00', 110.72, 116.37, 4)
('2026-08-03 20:00', 107.94, 117.25, 4)
('2026-08-03 19:00', 82.48, 89.86, 4)
```

Keeping MAX alongside AVG in every window query costs nothing and is the cheapest defence you have against the failure in the next step.

B6 The number you write down

CODE

```
peak = con.execute("""SELECT MAX(latency_ms) FROM kpi_readings
  WHERE cell_id='CELL-DEL-03' AND reading_ts LIKE '2026-08-03%'""").fetchone()[0]

four_h = con.execute("""SELECT MAX(a) FROM (
  SELECT AVG(latency_ms) a FROM kpi_readings
  WHERE cell_id='CELL-DEL-03' AND reading_ts LIKE '2026-08-03%'
  GROUP BY CAST(substr(reading_ts,12,2) AS INTEGER)/4)""").fetchone()[0]

raw = con.execute("""SELECT COUNT(*) FROM kpi_readings
  WHERE cell_id='CELL-DEL-03' AND reading_ts LIKE '2026-08-03%'
  AND latency_ms > 100""").fetchone()[0]

print(f"true 15-minute peak              : {peak:.2f} ms")
print(f"highest 4-hour mean               : {four_h:.2f} ms")
print(f"understated by                   : {(peak-four_h)/peak*100:.1f}%")
print(f"breaches of 100 ms on the raw grain: {raw}")
print(f"breaches of 100 ms on the aggregate: {int(four_h > 100)}")
```

EXPECTED OUTPUT

```
true 15-minute peak              : 117.25 ms
highest 4-hour mean              : 76.22 ms
understated by                   : 35.0%
breaches of 100 ms on the raw grain: 8
breaches of 100 ms on the aggregate: 0
```

THIS IS THE FINDING 8 genuine breaches on the raw grain. Zero on the four-hour aggregate. Nobody deleted anything - the quiet hours either side of the incident averaged it out of existence. Alert near the raw grain; aggregate only for reporting. Every KPI where the peak is the thing that hurts - latency, drop rate, congestion - obeys this rule.

B7 Build a rule that a NOC would actually keep switched on

Start with a rule that is correct and unusable, then measure exactly how unusable it is. The measurement is the exercise - a tuned rule you cannot show a before-and-after count for is a guess.

CODE

```
THRESHOLD = 70     # ms

# stage 0 - the naive rule on the raw file
raw_alerts = ((kp.latency_ms > THRESHOLD) | (kp.downtime_min > 0)).sum()

# stage 1 - the same rule on the de-duplicated table
dedup_alerts = ((kc.latency_ms > THRESHOLD) | (kc.downtime_min > 0)).sum()

# stage 2 - drop anything inside a maintenance window
supp = ((kc.latency_ms > THRESHOLD) | (kc.downtime_min > 0)) & ~in_maint

# stage 3 - require three consecutive intervals (45 minutes)
fire = kc[supp].sort_values(["cell_id","ts"])
runs = []
for cid, g in fire.groupby("cell_id"):
    t = g.ts.tolist(); start = t[0]; prev = t[0]; n = 1
    for x in t[1:]:
        if (x - prev) == pd.Timedelta(minutes=15): n += 1
        else: runs.append((cid, start, n)); start = x; n = 1
        prev = x
    runs.append((cid, start, n))
long_runs = [r for r in runs if r[2] >= 3]

print("raw rule on the raw file  :", int(raw_alerts))
print("after de-duplication      :", int(dedup_alerts))
print("after maintenance suppression:", int(supp.sum()))
print("after 3-interval persistence :", sum(r[2] for r in long_runs))
print("after hysteresis grouping    :", len(long_runs))
for cid, start, n in sorted(long_runs, key=lambda r: -r[2]):
    print(f"   {cid}  from {start}  ({n} intervals)")
```

EXPECTED OUTPUT

```
raw rule on the raw file  : 75
after de-duplication      : 63
after maintenance suppression: 43
after 3-interval persistence : 27
after hysteresis grouping    : 3
  CELL-DEL-03  from 2026-08-03 19:00:00  (13 intervals)
  CELL-CHE-02  from 2026-08-04 19:15:00  (7 intervals)
  CELL-HYD-01  from 2026-08-02 09:15:00  (7 intervals)
```

Stage	Alerts	What was removed
Raw rule on the raw file	75	-
After de-duplication	63	12 duplicate breaching readings
After maintenance suppression	43	20 planned-outage readings
After 3-interval persistence	27	15 short runs that self-healed
After hysteresis grouping	3	repeat notifications for the same condition

WHAT THE STAGES ARE DOING De-duplication is the first stage of alert reduction, not merely a data-cleaning nicety - a double-writing collector inflates your alert volume before you have tuned anything.
Suppression is not about false positives. The cell really was down. It is worse than a false positive: you have paged an engineer to tell them about something they are currently doing.
15 of the 18 breach runs in this feed are shorter than three intervals. Every one of them would have paged somebody, and every one was over before anyone could respond.

IF YOU FINISH WITH FEWER THAN THREE Your mute timing is too wide. Check whether CELL-DEL-03 is still firing. Suppressing a real incident along with the planned ones is the failure this step is designed to make you feel - and the reason your mute timing must be scoped to the three cells named in the maintenance file, not to every cell in that time range.

B8 Forecast, and be honest about the horizon

Thirty months of history per cell, and a capacity figure against each. Forecast six months forward and find the cells that run out of headroom. Prophet is the tool from Exercise 7 if it is installed on your machine; the code below needs nothing beyond pandas and numpy, so it will run anywhere.

CODE

```
t = pd.read_csv(f"{DATA}/capstone_cell_traffic_monthly.csv")
t["Month"] = pd.PeriodIndex(t.Month, freq="M").to_timestamp()
t = t.sort_values(["Cell","Month"])
t["Traffic_GB"] = t.groupby("Cell").Traffic_GB.transform(
                  lambda x: x.interpolate().bfill().ffill())

def forecast_cell(g, horizon=6):
    g = g.sort_values("Month").reset_index(drop=True)
    y = np.log(g.Traffic_GB.to_numpy()); x = np.arange(len(y))
    slope, intercept = np.polyfit(x, y, 1)          # trend in log space
    resid = y - (intercept + slope * x)
    seas  = pd.Series(resid).groupby(g.Month.dt.month.to_numpy()).mean()
    fx = np.arange(len(y), len(y) + horizon)
    fm = pd.date_range(g.Month.iloc[-1] + pd.offsets.MonthBegin(1),
                       periods=horizon, freq="MS")
    yhat = intercept + slope * fx + np.array(
            [seas.get(m.month, 0.0) for m in fm])
    return pd.DataFrame({"Cell": g.Cell.iloc[0], "Month": fm,
                         "Forecast_GB": np.round(np.exp(yhat), 1),
                         "Capacity_GB": g.Capacity_GB.iloc[-1]})

fc = pd.concat([forecast_cell(g) for _, g in t.groupby("Cell")],
               ignore_index=True)
fc["util_pct"] = (fc.Forecast_GB / fc.Capacity_GB * 100).round(1)

today = t.groupby("Cell").tail(1)
util_now = (today.set_index("Cell").Traffic_GB /
            today.set_index("Cell").Capacity_GB * 100).round(1)
print("cells above 85% utilisation today:", (util_now > 85).sum())
print(util_now.nlargest(5).to_string())
```

EXPECTED OUTPUT

```
cells above 85% utilisation today: 3
CELL-HYD-03   (see your own output for the exact figure)
CELL-KOL-03   (see your own output for the exact figure)
CELL-MUM-01   (see your own output for the exact figure)
```

YOU MAY NOT FIND A CRISIS, AND THAT IS A RESULT On this data the cells at risk are the ones already close to their limit today; the six-month forecast does not push a large new group over the line. A capacity case built on cells that are already saturated is a perfectly good case. Do not manufacture a dramatic crossing that the numbers do not support - and note in your write-up how much confidence a six-month horizon on thirty months of history actually deserves.

B9 Connect degradation to money

A cell serving a high-ARPU area is not the same business problem as one serving a low-ARPU area with identical latency. Join your cell-level quality to the subscriber file through home_cell_id and rank on exposure, not on milliseconds.

```
CODE
exposure = (s.groupby("home_cell_id")
            .agg(subs=("customer_id","count"),
                 mean_arpu=("arpu","mean"))
            .assign(monthly_revenue=lambda d: d.subs * d.mean_arpu)
            .reset_index().rename(columns={"home_cell_id":"cell_id"}))

quality = (kc.groupby("cell_id")
           .agg(p95_latency=("latency_ms", lambda x: x.quantile(0.95)),
                downtime_intervals=("downtime_min", lambda x: (x > 0).sum()))
           .reset_index())

risk = quality.merge(exposure, on="cell_id")
risk["revenue_at_risk"] = risk.monthly_revenue * (risk.p95_latency > 70)
print(risk.sort_values("revenue_at_risk", ascending=False).head(8).to_string(index=False))
```

THE QUESTION WE WILL ASK You suppress alerts during planned maintenance windows. A genuine fault begins fifteen minutes into one of those windows. What does your system do, when does anybody find out, and is that acceptable?

Track C - Customer care copilot

The business question: thousands of messages arrive every day. Which ones matter, who should handle them, and what is that worth?

Files: capstone_messages.csv, capstone_tickets.csv, and capstone_subscribers.csv

C1 Join the words to the customer

A message on its own tells you how somebody feels. A message joined to the subscriber file tells you how much that feeling is worth. Do the join first, and decide explicitly what happens to messages that do not match anybody.

```
CODE
m = pd.read_csv(f"{DATA}/capstone_messages.csv")
print("messages:", len(m))
print(m.true_sentiment.value_counts().to_string())

j = m.merge(s[["customer_id","arpu","tenure_months","region","plan_type"]],
            on="customer_id", how="left")
unmatched = j.arpu.isna().sum()
print("messages with no matching subscriber:", unmatched)
```

```
EXPECTED OUTPUT
messages: 354
Negative    152
Positive    107
Neutral      95
messages with no matching subscriber: 18
```

DO NOT SILENTLY DROP THE UNMATCHED Those 18 messages are real customers writing in. They may be prospects, wrong numbers, or a broken identity join upstream. Keep them, mark them, and decide what your queue does with a message whose value you cannot estimate - then say which choice you made and why.

C2 A sentiment reader you can see inside

Build the transparent version first. It counts positive and negative words and nothing else, which means you can always answer the question "why did it score that?" - a property worth more in an operations setting than a few points of accuracy.

CODE

```
POSITIVE = {"amazing","happy","great","love","easy","quick","excellent",
            "satisfied","best","thank","well","correct","pleased",
            "fantastic","good","fair","fast","minutes","solved"}
NEGATIVE = {"terrible","drop","drops","overcharged","unacceptable","no",
            "awful","worst","waited","nothing","disappointed","frustrated",
            "failed","slow","expensive","porting","unresolved","not"}

def score_message(text):
    words = text.lower().replace(",", " ").split()
    pos = sum(w in POSITIVE for w in words)
    neg = sum(w in NEGATIVE for w in words)
    if pos > neg: return "Positive", pos - neg
    if neg > pos: return "Negative", pos - neg
    return "Neutral", 0

from sklearn.metrics import accuracy_score
m[["predicted","score"]] = m.message_text.apply(
    lambda t: pd.Series(score_message(t)))

print("accuracy, all messages :",
      round(accuracy_score(m.true_sentiment, m.predicted)*100, 1), "%")
for grp, g in m.groupby("case_type"):
    print(f"accuracy, {grp:9} cases: "
          f"{accuracy_score(g.true_sentiment, g.predicted)*100:5.1f} %"
          f"   (n={len(g)})")
```

EXPECTED OUTPUT

```
accuracy, all messages : 92.1 %
accuracy, hard      cases:  25.0 %   (n=24)
accuracy, standard  cases:  97.0 %   (n=330)
```

LOOK AT THE TWO NUMBERS SEPARATELY 97.0 per cent on ordinary messages is genuinely good for a word list you could write in five minutes. 25.0 per cent on the hard cases is a collapse.

The 24 hard cases are marked in the case_type column. Read them. They contain negation ("the staff were not helpful at all"), litotes ("the network is not bad actually") and sarcasm. A word counter sees "helpful" and "bad" and gets all of them backwards. Report the headline figure and the breakdown - quoting only the 92.1 per cent average would hide the entire finding.

C3 The comparison, if your machine allows it

If a transformer model has been pre-staged on your machine, run the same hard cases through it and put the two side by side. If it has not, say so in your notebook and reason about the trade-off from the properties instead - that is a legitimate answer and we would rather have it than a fabricated result.

CODE

```
# OPTIONAL - requires transformers + torch, pre-installed and cached.
# If this cell fails, note it in the notebook and move on to C4.
from transformers import pipeline
sentiment_ai = pipeline("sentiment-analysis",
      model="distilbert-base-uncased-finetuned-sst-2-english")

hard = m[m.case_type == "hard"].drop_duplicates("message_text")
for _, r in hard.iterrows():
   ours, _ = score_message(r.message_text)
   ai = sentiment_ai(r.message_text)[0]["label"].title()
   print(f"{r.message_text[:46]:48} true={r.true_sentiment:9}"
        f" ours={ours:9} model={ai}")
```

BE HONEST ABOUT WHERE IT ALSO FAILS The transformer handles negation well and will beat the word counter on most of the hard cases. It will still get the sarcastic ones wrong, and it has no Neutral class at all, so plain requests get forced onto one side or the other. There is no tool here that solves everything, and a slide claiming otherwise is the easiest thing for us to take apart.

C4 Route the ticket, and measure how sure you are

CODE

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

tk = pd.read_csv(f"{DATA}/capstone_tickets.csv")
print("tickets:", len(tk), " teams:", tk.team.nunique())

vec = TfidfVectorizer()
X = vec.fit_transform(tk.ticket_text)
y = tk.team
print("each ticket is now", X.shape[1], "number-columns")

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
                                      random_state=42, stratify=y)
router = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
pred = router.predict(Xte)
print("routing accuracy:", round(accuracy_score(yte, pred)*100, 1), "%")
```

EXPECTED OUTPUT

```
tickets: 548  teams: 5
each ticket is now 195 number-columns
routing accuracy: 88.3 %
```

WHY IT IS NOT HIGHER, AND WHY THAT IS REALISTIC Some tickets in this file were sent to a team the words alone do not point to - because a human router knew about an open case, an account history or a supervisor instruction that the message never carried. No model can recover information that is not in the text. That gap is not a defect in your model; it is the reason the next step exists.

C5 Set a confidence floor from the data, not from a round number

Everyone reaches for eighty per cent. Look at what eighty per cent actually costs you here before you commit to it.

CODE

```
conf = router.predict_proba(X).max(axis=1)
for floor in [0.5, 0.6, 0.7, 0.8, 0.9]:
    share = (conf < floor).mean() * 100
    print(f"floor {floor:.0%}: {share:5.1f}% of all tickets go to a human")
print("median confidence:", round(float(np.median(conf)), 3))

amb = tk[tk.case_type == "ambiguous"]
ca  = router.predict_proba(vec.transform(amb.ticket_text)).max(axis=1)
print("ambiguous tickets:", len(amb),
      " mean confidence:", round(float(ca.mean()), 3))
```

EXPECTED OUTPUT

```
floor 50%:   4.9% of all tickets go to a human
floor 60%:  14.4% of all tickets go to a human
floor 70%:  42.9% of all tickets go to a human
floor 80%:  86.3% of all tickets go to a human
floor 90%: 100.0% of all tickets go to a human
median confidence: 0.713
ambiguous tickets: 36  mean confidence: 0.624
```

THE DECISION IS YOURS AND IT IS NOT OBVIOUS An eighty per cent floor sends 86.3 per cent of tickets to a human, which automates almost nothing and is not worth deploying. A fifty per cent floor sends 4.9 per cent, which automates nearly everything including the tickets the model is least sure about. State your floor, state what it costs in human review time, and state what it lets through. A floor that catches nothing is decoration; a floor that catches most of your traffic is not automation.

C6 Rank the queue on anger and value together

CODE

```
j["is_negative"] = (j.message_text.apply(lambda t: score_message(t)[0])
                    == "Negative").astype(int)
j["anger"] = -j.message_text.apply(lambda t: score_message(t)[1])
j["priority"] = j.anger * j.arpu.fillna(j.arpu.median())

queue = j.sort_values("priority", ascending=False)
print(queue[["message_id","customer_id","arpu","anger",
             "priority","message_text"]].head(10).to_string(index=False))
queue.head(50).to_csv("escalation_list.csv", index=False)
```

THE QUESTION WE WILL ASK Your router sends a ticket to the wrong team at 84 per cent confidence, and your floor is set at 80 per cent. Whose fault is that - the model, the floor, or the person who chose the floor? And what do you change on Monday?

6. Part 3 - Turn the analysis into a decision

Every team does this part. It is the step none of the exercises did for you, and it is where the project is won or lost. An analysis that stops at a finding has not finished.

6.1 A dashboard a non-analyst could read

Grafana or Power BI, your choice. Four panels maximum. Every panel must answer a question you can say out loud in one sentence - if nobody on the team can state the question, delete the panel.

Track	A panel set that works
A	Churn risk by region and plan; ARPU against risk; your target segment isolated; expected loss by segment
B	Latency by cell over time; throughput by cell; cells currently in downtime; worst cells right now
C	Queue volume by team; negative share over time; confidence distribution; the escalation list

> **CAUTION** The aggregation you use for a wallboard is not the aggregation you use for an alert. You have the number in front of you: on CELL-DEL-03 the true fifteen-minute peak was 117.25 ms while the highest four-hour mean was 76.22 ms - the same data, the same day, the incident understated by 35.0 per cent. Say which grain each panel uses and why that grain is right for that question.

6.2 Three options, one of which is doing nothing

Take the group your analysis says matters most. Propose three courses of action and put a number against each. One of the three must be doing nothing - without it you have no reference point and no way to show that acting is better than not acting.

CODE

```
segment_size = ...       # subscribers in your target group
base_churn   = ...       # from your model, or your segment profile
value_lost   = 5880      # lifetime value of a lost subscriber

options = [
    {"name": "Do nothing",         "cost_per_sub": 0,   "reduction": 0.00},
    {"name": "Retention call",     "cost_per_sub": 300, "reduction": 0.15},
    {"name": "Bill credit + call", "cost_per_sub": 550, "reduction": 0.25},
]

for o in options:
    saved    = segment_size * base_churn * o["reduction"]
    spend    = segment_size * o["cost_per_sub"]
    o["net"] = saved * value_lost - spend
    print(f'{o["name"]:22} saves {saved:6.0f} subs   '
          f'spend Rs {spend:>10,.0f}   net Rs {o["net"]:>12,.0f}')
```

> **THE NUMBERS IN THAT SCAFFOLD ARE ASSUMPTIONS** Nothing in your data tells you how well a retention call works. The 15 and 25 per cent figures are placeholders - replace them with your own and name them as assumptions when you present.
> Then halve the most important one and show what happens. A recommendation that survives that test is worth acting on. One that does not is worth knowing about before you stand up, not after.
> Do not be alarmed if doing nothing wins on your numbers. That is a real result and a much more interesting slide than a comfortable one.

6.3 Close in the SIAV structure

Part	Means	Example
Situation	The business problem, in their words	We are losing subscribers and finding out too late
Insight	What your analysis found	One segment carries 1.7 times the churn risk of the base
Action	What you want done	Route that segment to proactive retention before renewal
Value	What it is worth, costed	Catching them is worth roughly ₹X a quarter, against ₹Y spent

7. The slide deck

Design it yourselves. The look, the flow and the visuals are your team's to decide. It must cover the following, but how you present each one is up to you. Aim for a deck a campaign manager or a network planner could follow without ever seeing your code.

- Title - the project, the team, and each person's area of ownership.
- The problem - what you set out to find, and why an operator would care.
- Your data - sources, grain, and what you repaired. One slide.
- Your model - with the honest baseline beside it, on the same slide.
- The central finding - shown visually, not described in bullets.
- The costed comparison - all three options, with the assumptions named.
- One recommendation - in the SIAV structure.
- Your honest limitation, and the one thing you would do next with another week.

8. Submission

One submission per team to the shared folder, before the start of the Day 10 session.

File	What it is
FinalCapstone_<team>.ipynb	All analysis, running top to bottom on a clean kernel with no manual steps
FinalCapstone_<team>_dataset.csv	Your integrated dataset, exactly as your pipeline produced it
FinalCapstone_<team>_dashboard.*	The Power BI file, or the exported Grafana dashboard JSON
FinalCapstone_<team>_deck.pptx	The presentation

Every team member's name goes on the title slide and in the first cell of the notebook.